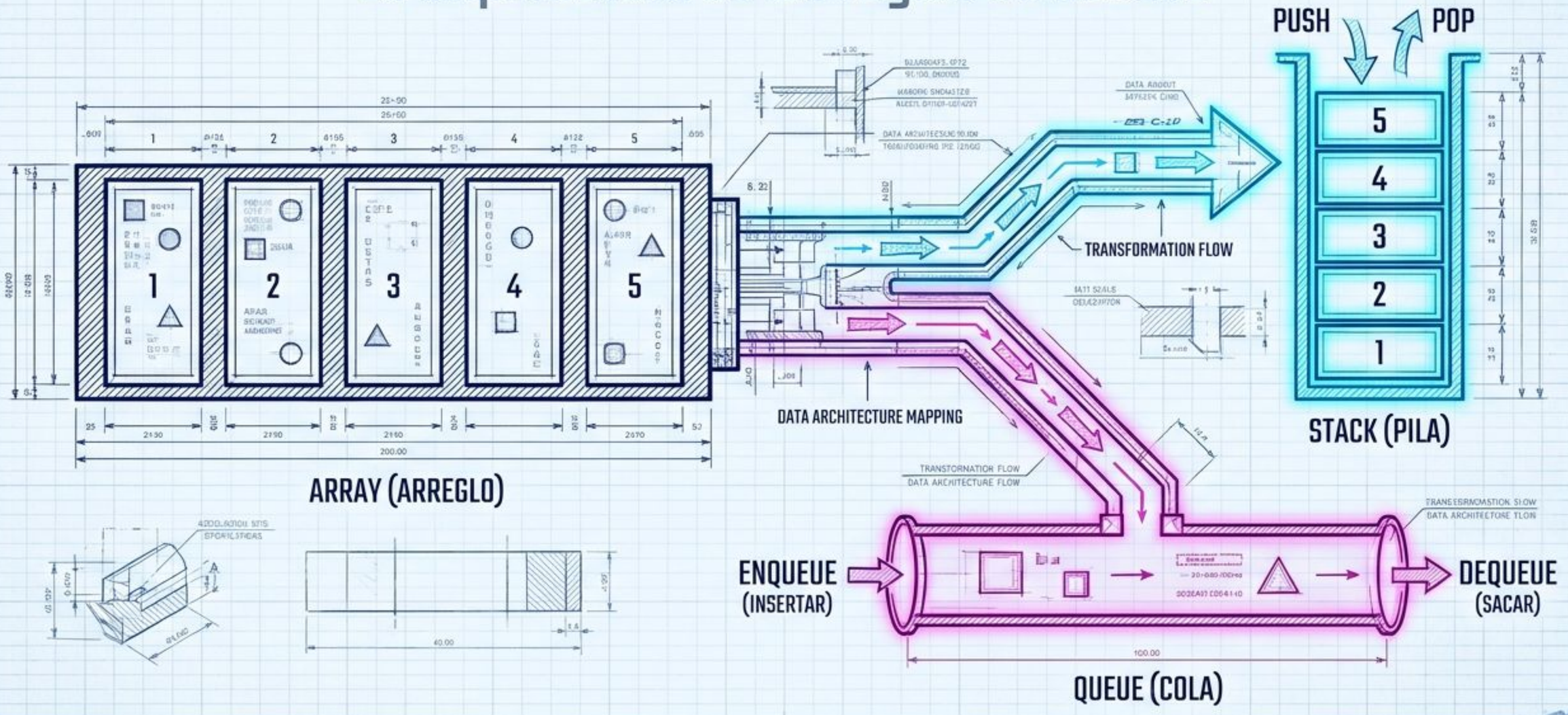
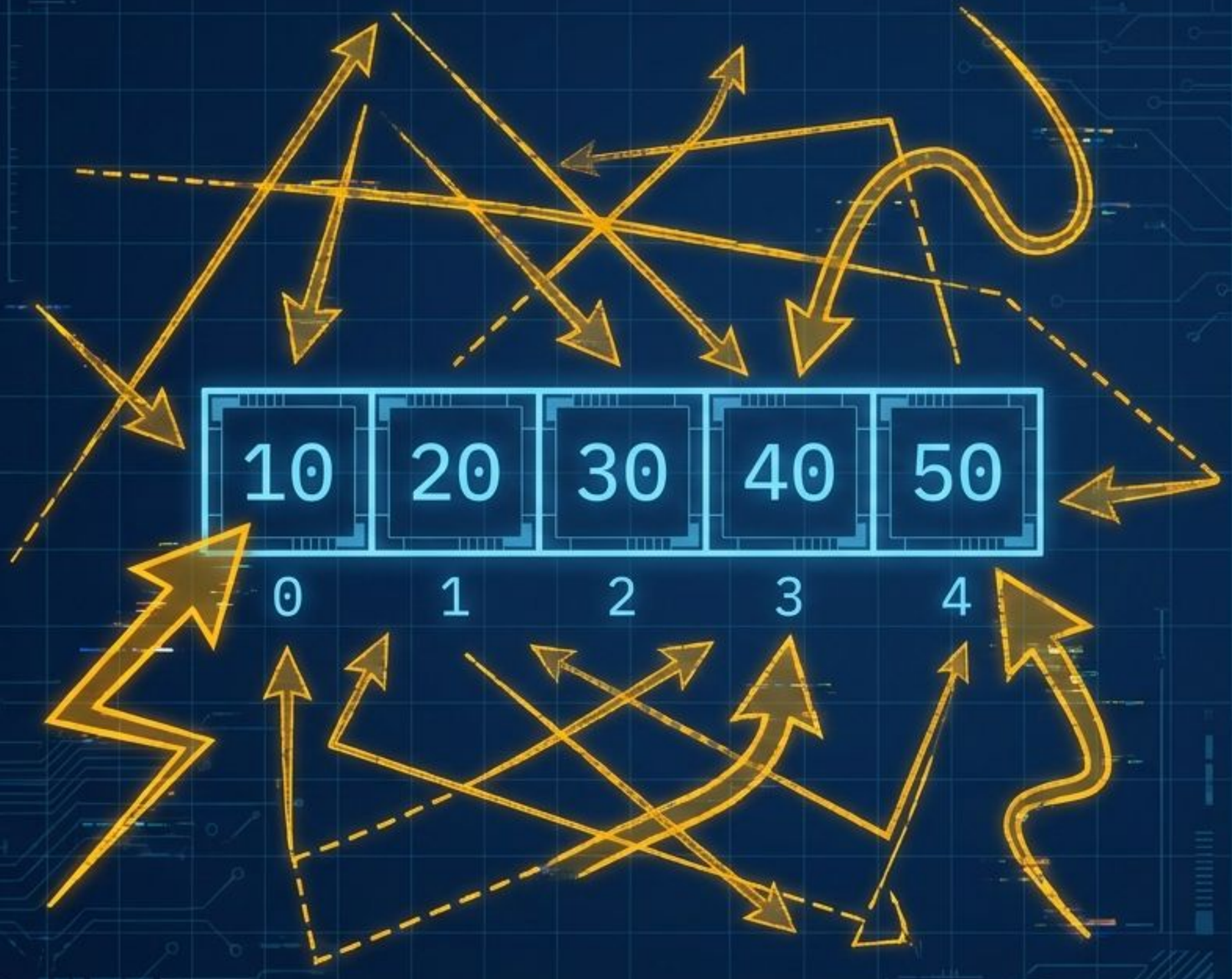
Estructuras de Datos: Pilas y Colas usando Arreglos

La arquitectura de las reglas de acceso



NotebookLM



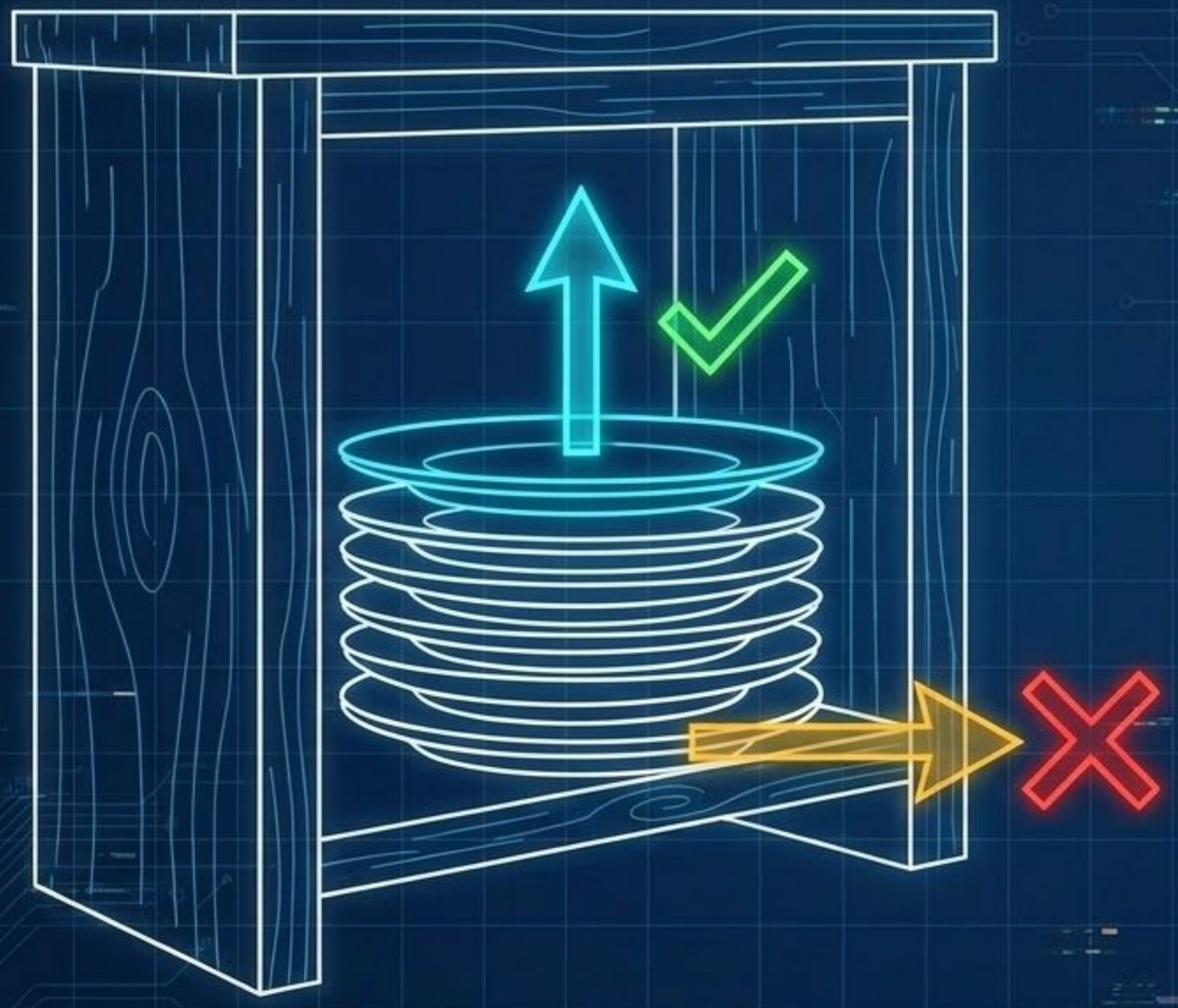
El caos de la libertad absoluta

Hasta ahora, hemos trabajado con arreglos tradicionales.

En un arreglo estándar, podemos acceder a cualquier posición en cualquier momento sin restricciones.

⚠ ¿Pero qué ocurre cuando las reglas del problema no permiten leer cualquier dato? En el software, el acceso irrestricto a menudo es un error lógico.

NotebookLM



A veces, el acceso total es físicamente imposible

Si queremos sacar el primer plato que guardamos, no podemos simplemente tirar desde abajo.

Las leyes de la física imponen un orden. Las reglas de nuestro problema nos obligan a interactuar únicamente con el plato superior.

No podemos acceder libremente a los datos porque la naturaleza del problema lo impide.

NotebookLM



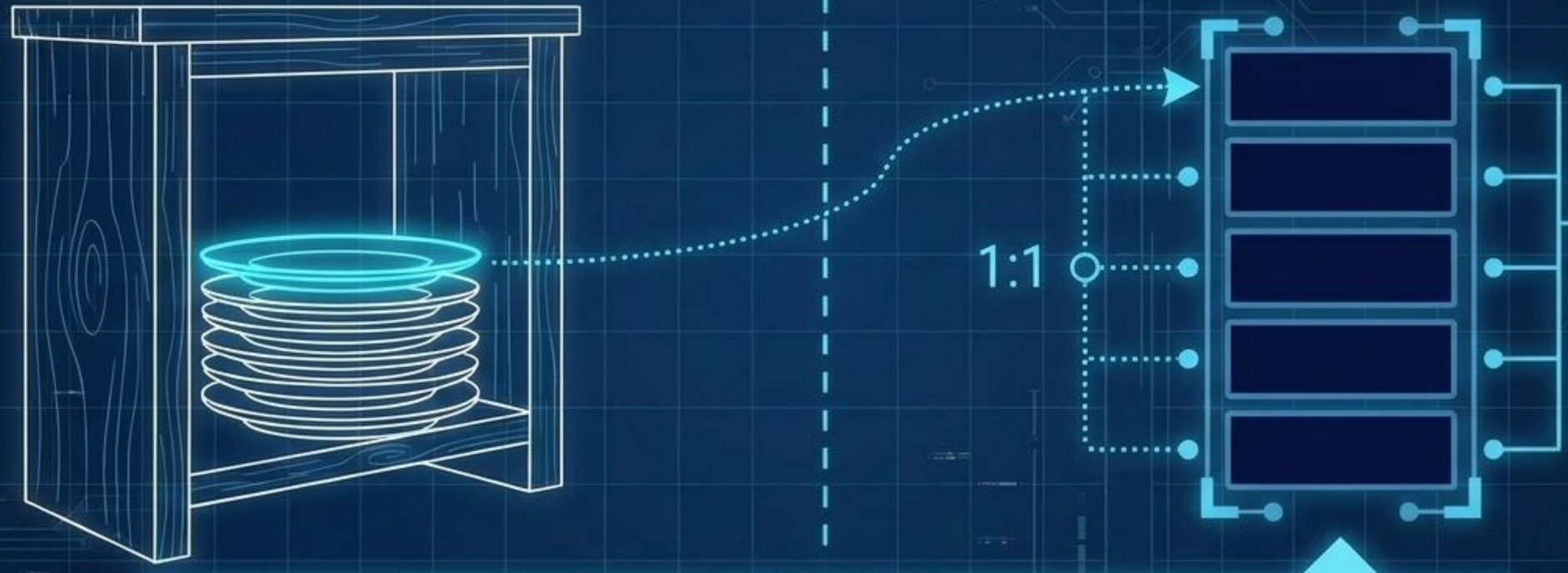
La fórmula de una Estructura de Datos

Una estructura de datos no es magia. Es simplemente una forma organizada de almacenar información, encapsulada y protegida por reglas de acceso inquebrantables.

NotebookLM

La Pila impone la regla del último

La Pila (Stack) es la estructura que nace de la necesidad de apilar información. Se rige por un principio arquitectónico innegociable: LIFO.



LIFO (Last In, First Out). El último elemento en entrar es obligatoriamente el primero en salir.

NotebookLM

Tres comandos para gobernar la Pila

Una pila bloquea el acceso directo y solo expone tres operaciones.



Push

Agrega un elemento a la cima.



Pop

Elimina y devuelve el elemento de la cima.

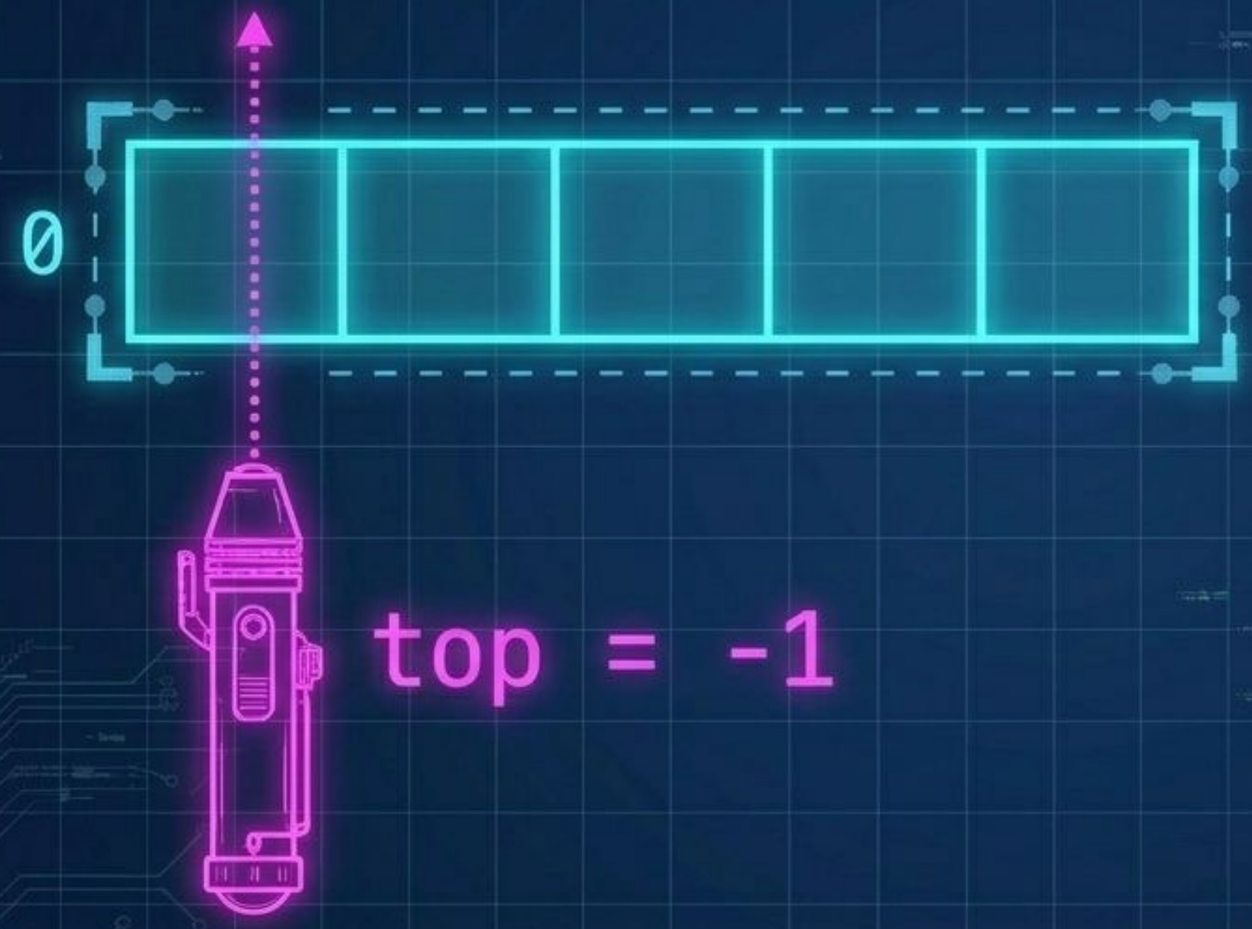


Top (Peek)

Consulta el elemento superior sin eliminarlo.

NotebookLM

El puntero Top: Nuestro láser de seguimiento



El puntero Top: Nuestro láser de seguimiento

Para implementar una Pila en un arreglo, necesitamos saber exactamente dónde está el último elemento.

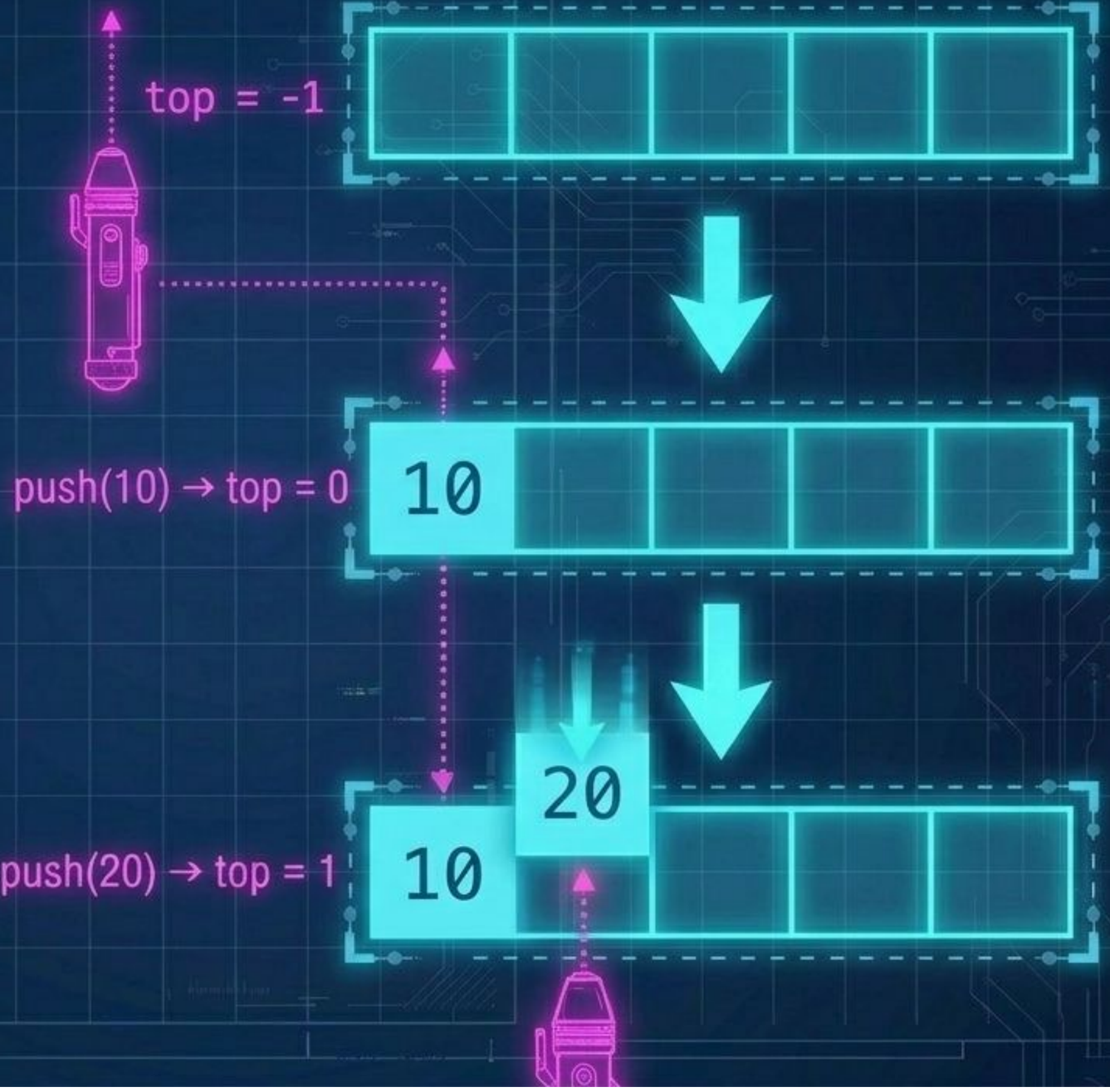
Usamos una variable puntero llamada `top`.

¿Por qué inicializamos `top = -1`? Porque el índice 0 ya representa una posición válida con datos. Si la pila está vacía, el puntero debe apuntar fuera de los límites del arreglo.

NotebookLM

La anatomía del comando Push

Insertar un dato requiere una coreografía estricta de dos pasos. Primero movemos el láser, luego colocamos el dato. (top++; seguido de pila[top] = valor;).



NotebookLM

La ilusión del Pop: Los datos fantasma

Al hacer pop(), simplemente retrocedemos el puntero (top--).



No borramos físicamente el elemento de la memoria, lo ignoramos lógicamente. El próximo push simplemente sobrescribirá a ese fantasma.

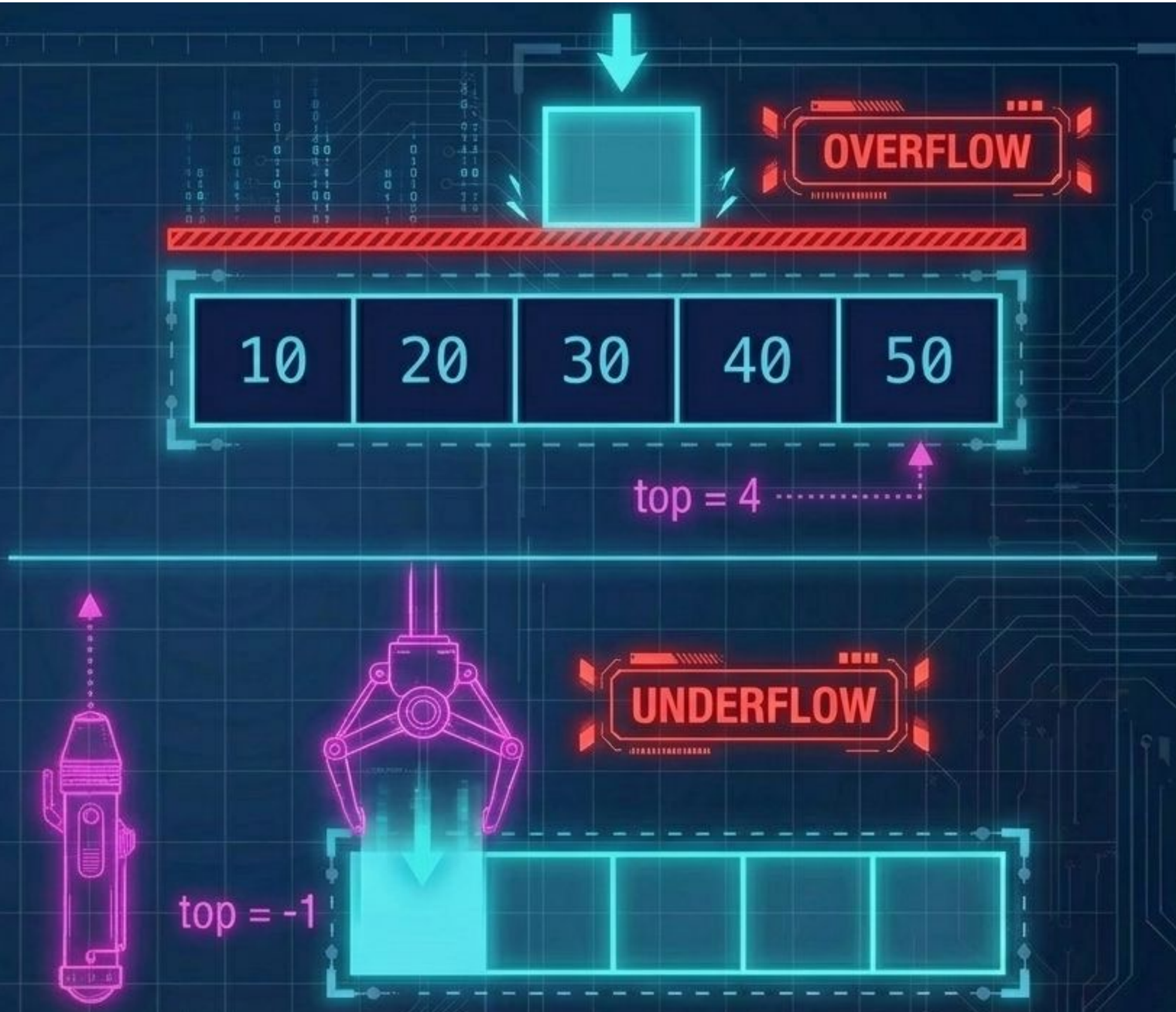
NotebookLM

Los límites de la estructura física

Debido a que la Pila se construye sobre un arreglo estático, heredamos sus fronteras físicas.

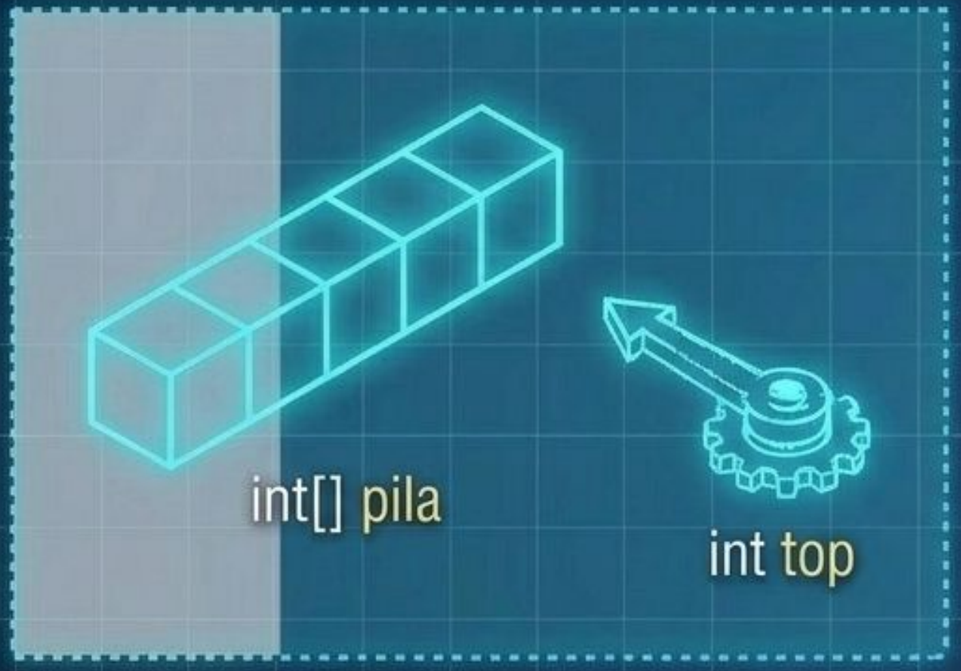
Overflow: Ocurre al intentar un push cuando top alcanzó el límite máximo del arreglo.

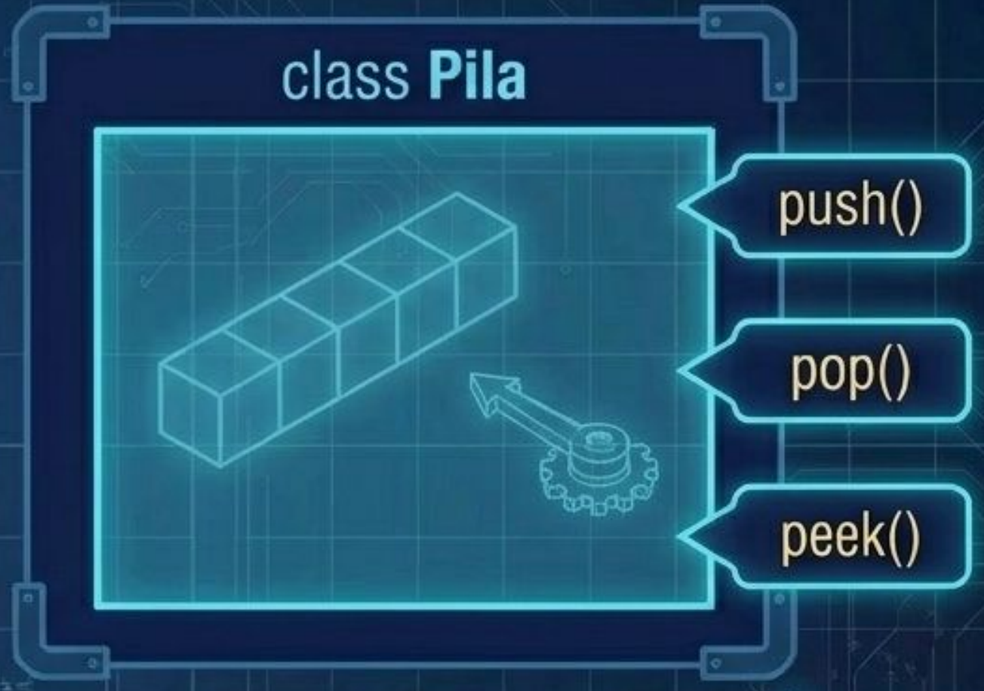
Underflow: Ocurre al intentar un pop cuando top == -1 (no hay elementos).



NotebookLM

Evolucionando a la encapsulación





Evolucionando a la encapsulación

Si dejamos las variables sueltas, cualquiera podría alterar top manualmente y romper las reglas LIFO.

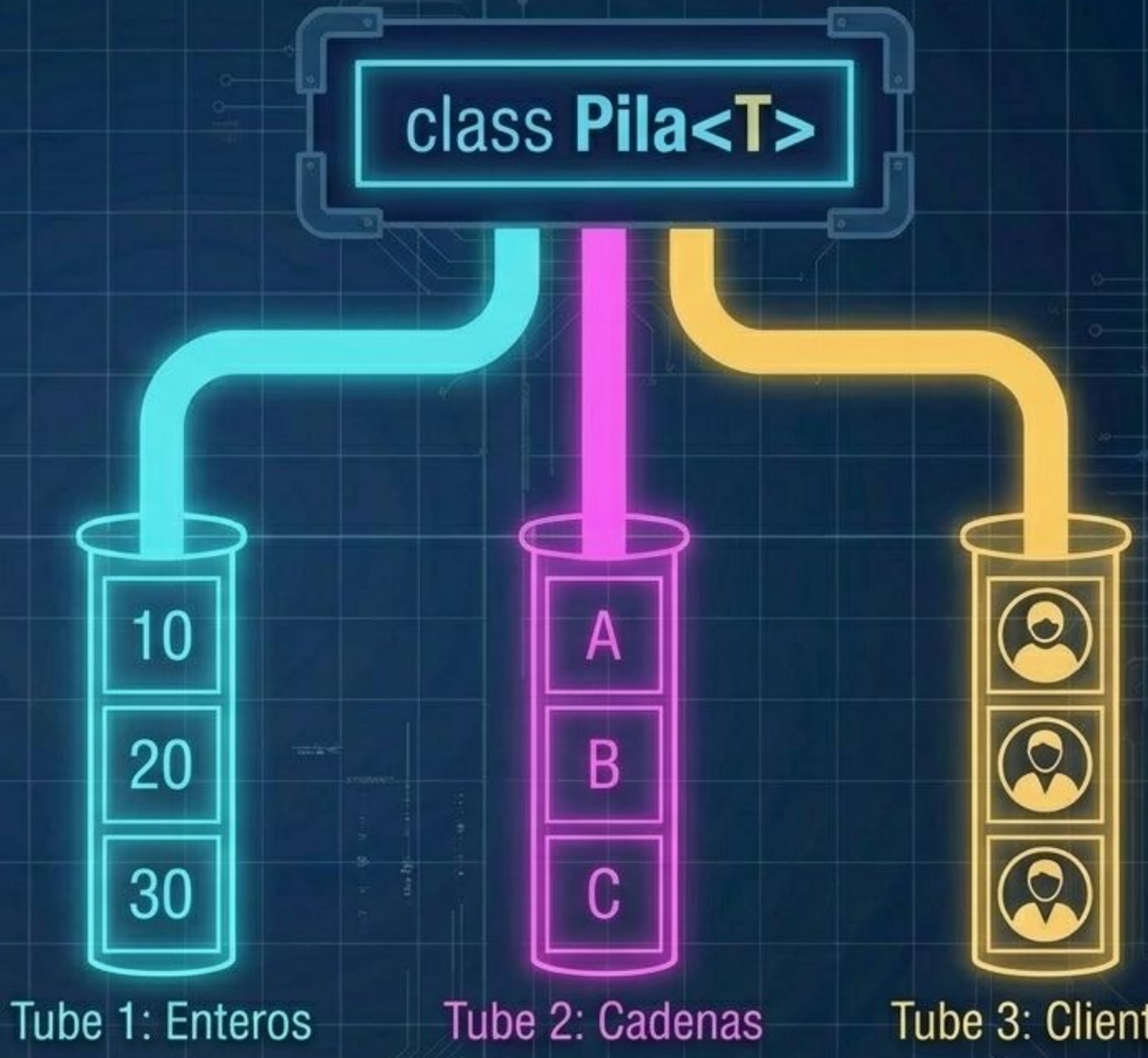
Encapsulamos el arreglo y el puntero dentro de una class Pila. Al ocultar la lógica interna, protegemos la estructura de datos y facilitamos su mantenimiento.

NotebookLM

El poder de la reutilización con Genéricos <T>

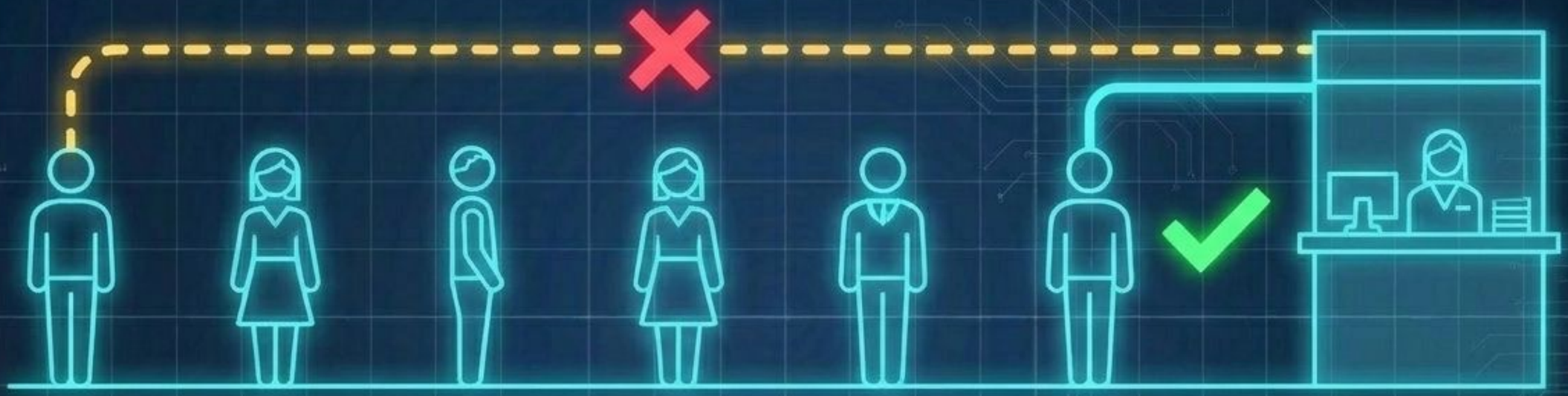
La regla LIFO es universal. No debería importar si apilamos números, palabras o perfiles de usuarios.

Al evolucionar nuestra clase a genérica, T actúa como un comodín arquitectónico, permitiendo reutilizar la misma lógica exacta para cualquier tipo de dato.



NotebookLM

UN NUEVO PROBLEMA EXIGE UNA NUEVA REGLA



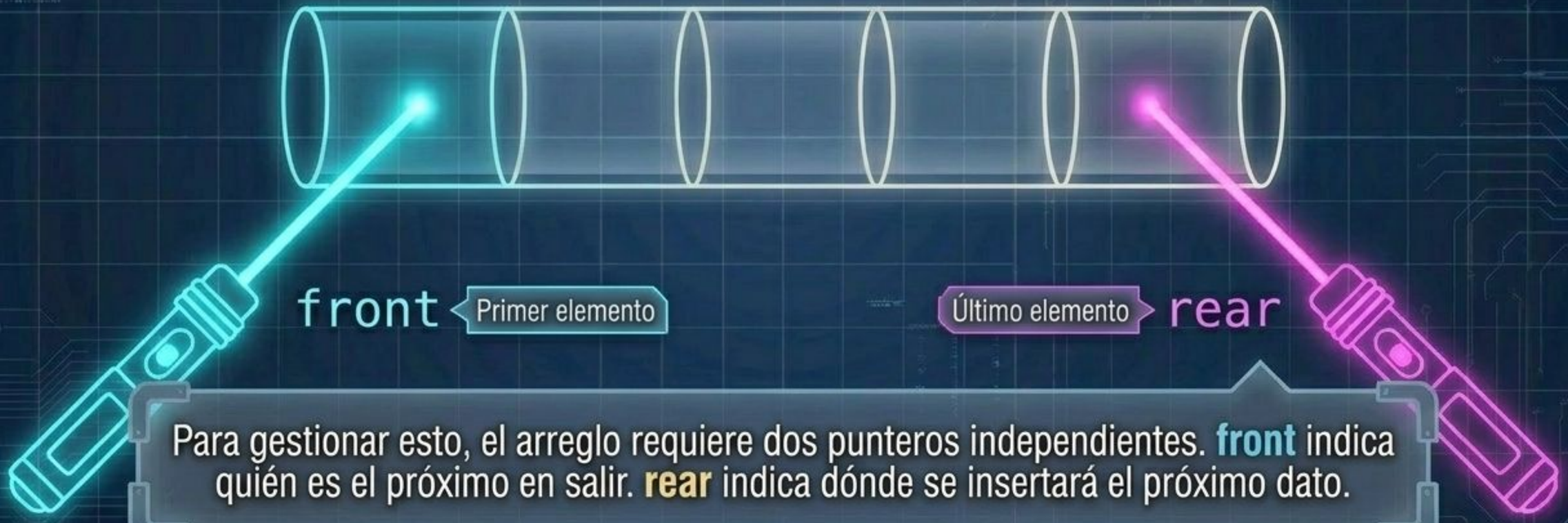
Si aplicamos una Pila a una fila de un banco, el primero en llegar sería el último en ser atendido. Eso es inaceptable.

Necesitamos una nueva estructura para garantizar la equidad: **La Cola (Queue)**.
Opera bajo la regla FIFO (First In, First Out). El primero que llega es el primero que sale.

NotebookLM

Controlando ambos extremos de la Cola

Una Pila solo necesitaba vigilar su cima. Una Cola tiene movimiento en ambos extremos. Los elementos entran por atrás y salen por adelante.



Para gestionar esto, el arreglo requiere dos punteros independientes. **front** indica quién es el próximo en salir. **rear** indica dónde se insertará el próximo dato.

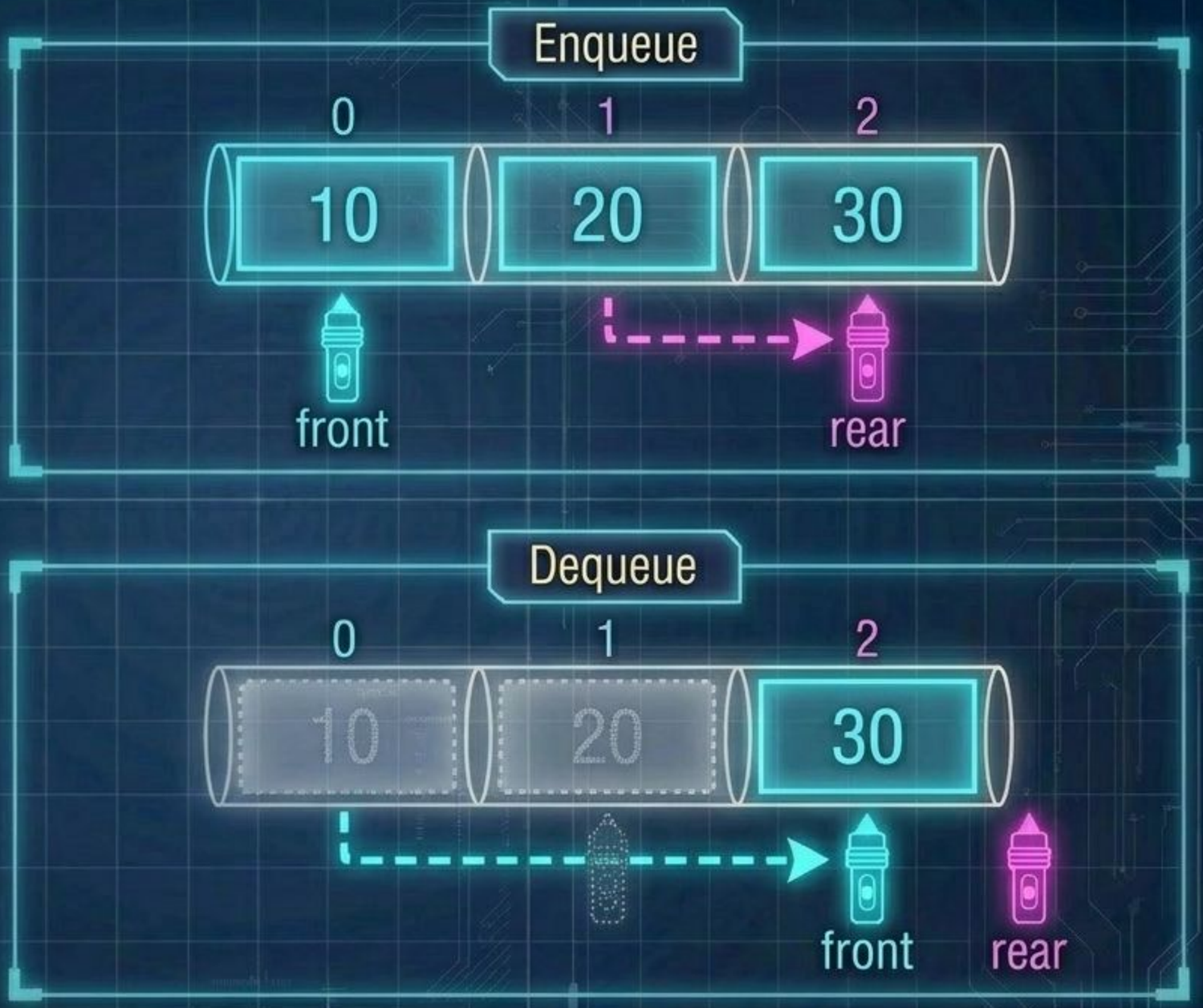
NotebookLM

Enqueue y Dequeue en acción

El flujo de datos se mueve hacia la derecha.

- Enqueue: Agrega datos. El puntero **rear** avanza.

- Dequeue: Elimina datos. El puntero **front** avanza persiguiendo a **rear**.



NotebookLM



El defecto oculto: El desperdicio de memoria

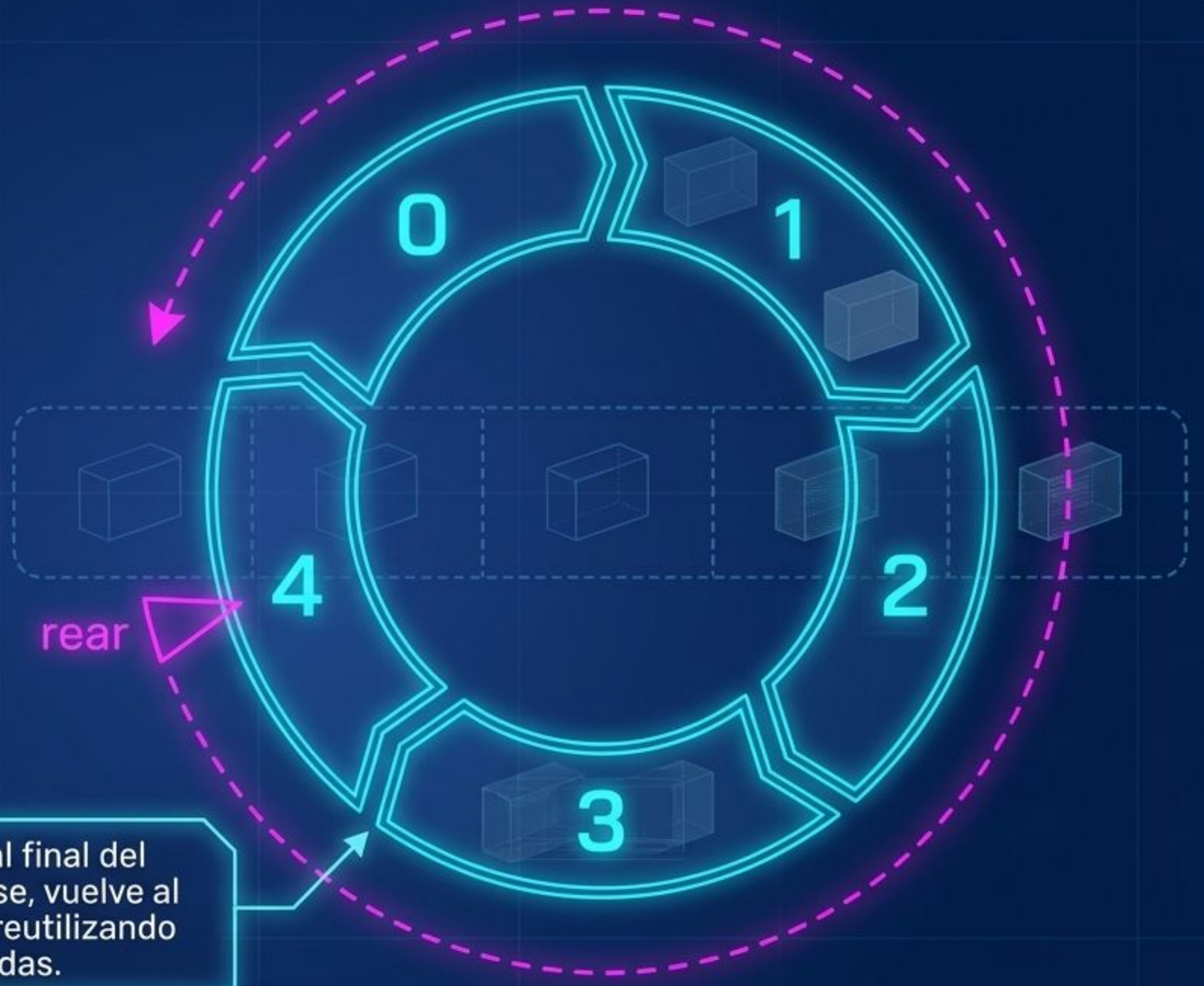
A medida que hacemos enqueue y dequeue, ambos punteros se desplazan hacia la derecha. Eventualmente, **rear** choca contra el final del arreglo.

El sistema marca "Overflow", impidiendo el ingreso de nuevos datos, ¡incluso cuando tenemos espacios perfectamente vacíos y liberados al frente!

NotebookLM

Doblando el espacio: La Cola Circular

Para solucionar el desperdicio, no reescribimos toda la arquitectura, simplemente cambiamos la geometría.



Cuando un puntero llega al final del arreglo, en lugar de detenerse, vuelve al principio de forma circular, reutilizando las posiciones liberadas.

NotebookLM

(Posición Actual + 1) % Tamaño = Siguiente Posición



(3 + 1) % 5 = 4

(4 + 1) % 5 = 0

La matemática detrás del anillo infinito

La magia de la Cola Circular se logra alterando una sola línea de código utilizando el operador Módulo (%).

El módulo nos da el resto de una división. Garantiza matemáticamente que los punteros nunca excedan los límites del arreglo, obligándolos a reiniciar en 0 cuando alcanzan el tamaño máximo.

NotebookLM

Matriz de Diagnóstico Arquitectónico

Ambas estructuras utilizan arreglos internamente. Su única y **radical diferencia** radica en las **reglas de acceso** que imponen.

Dimensión	Pila (Stack)	Cola (Queue)
Regla	LIFO	FIFO
Punteros Necesarios	1 (top)	2 (front, rear)
Metáfora Física	Torre de platos	Fila del banco
Comportamiento al Eliminar	Se saca el último agregado	Se saca el más antiguo

NotebookLM

La arquitectura invisible del mundo real

Estas estructuras operan silenciosamente en cada dispositivo moderno.

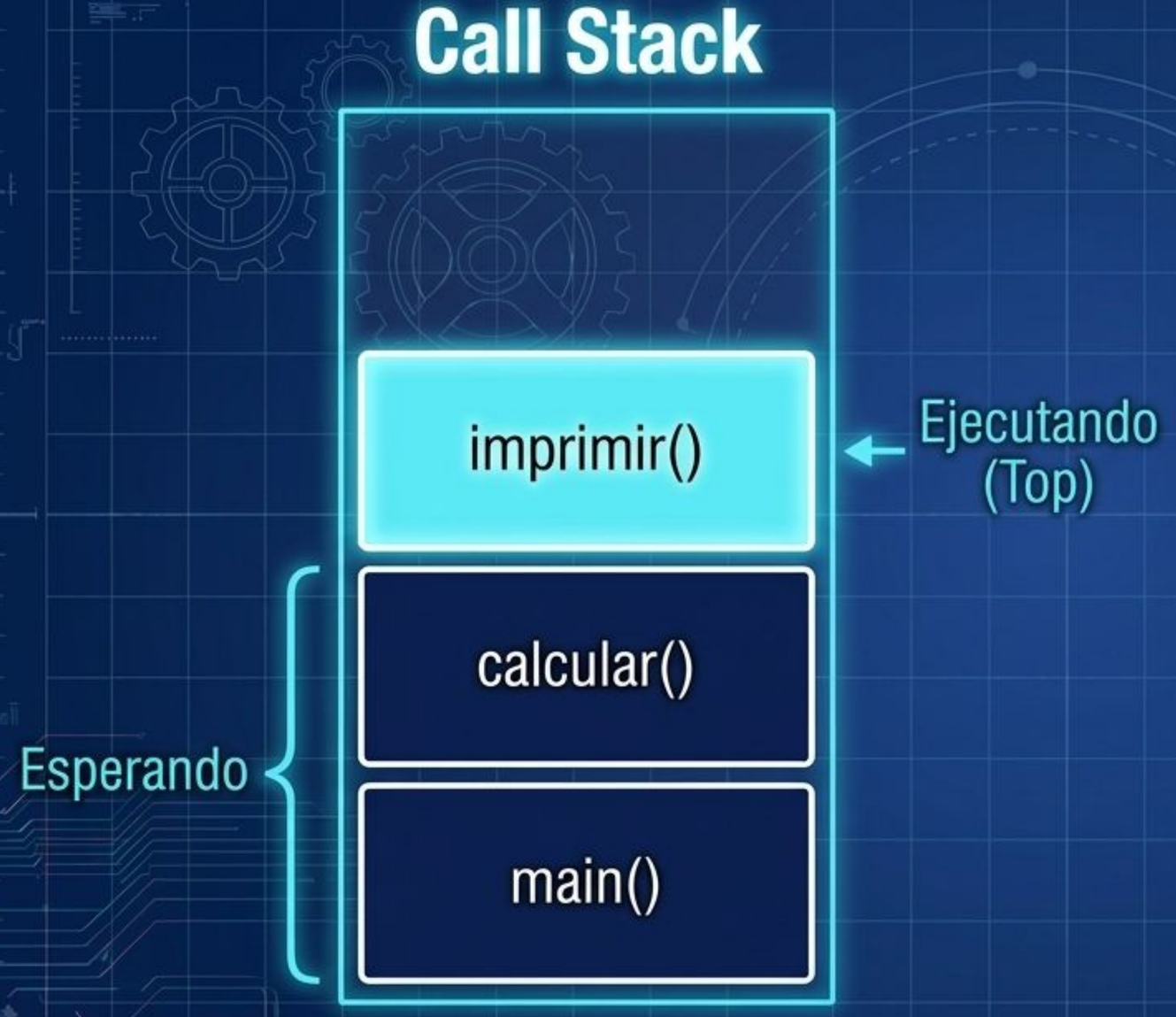
Pilas en Acción

- Historial de “Deshacer” (Undo/Redo)
- Evaluación de expresiones en compiladores
- Navegación de “Atrás” en el navegador

Colas en Acción

- Trabajos en cola de la impresora
- Atención de peticiones en servidores web
- Asignación de procesos por el sistema operativo

NotebookLM



La conexión final: La Pila de Llamadas

¿Recuerdas la recursividad? La espera sucesiva de las funciones no ocurre por arte de magia. Internamente, Java y otros lenguajes utilizan un Call Stack para gestionar los marcos de ejecución.

Las estructuras de datos no son solo arreglos con nombres distintos; son las reglas maestras que sostienen toda la lógica digital.

NotebookLM